

Important Questions:-

1. **Class :-** In Object Oriented Programming, a class is an extensible program-code-template for creating objects, providing initial values for state (member variables) and implementation of behavior (member functions or methods). The default access specifier in c# is **Internal**. Class is a **reference type**.
 - Abstract Class (Pure Virtual Class)
 - Partial Class
 - Sealed Class
 - Static Class
2. **Object :-** An Object is basically a block of memory that has been allocated and configured according to the blueprint. A program may create many objects of the same class. Objects are also called instances, and they can be stored in either a named variable or in an array of collections. In the simple words "**An Object is an unordered list of primitive data types (and sometimes reference types) that is stored as a series of name-value pairs. Each item in the list is called property (functions are methods)**".
3. **Abstract Class :-** The dictionary meaning of the word '**abstract**' is thought or idea that has no physical existence and is just conceptual. We can use the idea to build something of a physical existence. In MSDN Library the '**abstract**' keyword indicates that the thing has a missing or incomplete implementation and must be complete by others.

An Abstract class is an incomplete class or special class we can't instantiate directly, and it is a compile-time error to use the new operator on an abstract class. While it is possible to have variables and whose compile-time types are abstract, such variable and values will necessarily either be null or certain reference of non-abstract classes derived from the abstract types. It's implement logic is provided by the classes that derive from it. It can have both abstract as well as non-abstract methods. It is not compulsory to have only abstract methods in an abstract class. We can also have an abstract class with only non-abstract method.

Key Points :-

- An abstract class can inherit from a class and one or more interface.
 - An abstract class can implement code with non-abstract method.
 - An abstract class can have modifiers for method. properties etc...
 - An abstract class can have constants and fields.
 - An abstract class can implement a property.
 - An abstract class can have constructors and destructors.
 - An abstract class cannot inherit from structure.
 - An abstract class cannot support multiple inheritance.
 - An abstract class cannot be sealed or static.
 - An abstract class method can be static and it can call by abstract class name and method.
 - We can not create an object of abstract class, but we create a reference of it.
 - An Inheritance between abstract to abstract classes is possible. We don't need to implement abstract methods of these base abstract class into a derived abstract class. we can implement it in concrete class
 - Abstract method cannot be marked virtual.
 - A concrete class cannot inherit more than one abstract class.
4. **Partial Class :-** A partial class allow a single class to be divided into two separate physical files. During compile time, these files get compiled into a single class. During compilation, these files get compiled into a single class internally.

Key Points :-

- Inheritance at any partial type applies to the entire class.
 - The partial type must have the same accessibility.
5. **Sealed Class :-** Sealed class are used to restrict the inheritance features of OOPS. Once a class is defined as a sealed class, the class cannot be inherited. A sealed class cannot be used as a base class. For this reason, it cannot be an abstract class. Sealed classes are primarily used to prevent derivation. Because they can never be used as a base class, some run-time optimizations can make calling sealed class members slightly faster. Sealing a class means one can not derive from it. Sealing a method means one can not override it. If we try to inherit from a sealed class in another class we will get compile time error about inconsistent accessibility
 6. **Static Class :-** A Static class is one which cannot be instantiated. The keyword **new** cannot be used with static class can be called directly by using the class name itself.

Key Points :-

- A Static class can only have static members.
- A Static class cannot be instantiated
- A Static class is sealed, so cannot be inherited.
- A Static class cannot have a constructor.

7. **Constructor :-** Constructor have the same name as the class or struct, and they usually initialize the data members of the new object.

A class or struct have multiple constructors that take different arguments. Constructor enables the programmer to set default values, limit instantiation. We cannot call constructors explicitly. Constructor are responsible for object initialization and memory allocation of it's class. If we create any class without constructor, the compiler will automatically create one default constructor for that class. There is always at least one constructor in every class. There are different types of constructor

- **Default Constructor** :- If a class contains no instance constructor declarations, a default instance constructor is automatically provided. That default constructor simply invokes the parameterless constructor of the direct base class. If the direct base class does not have an accessible parameterless instance constructor, a compile-time error occurs. If the class is abstract then the declared accessibility for the default constructor is protected. Otherwise, the declared accessibility for the default constructor is public. Thus the default constructor is always of the form.
- **Copy Constructor** :- A parameterized constructor that contains a parameter of same class type is call as copy constructor. Main purpose of copy constructor is to initialize new instance to the values of an existing instance. We need copy constructor to initialize one object from other object, and generally we use assignment operator to invoke copy constructor.
- **Private Constructor** :- A private constructor means that the class can't be instantiated from outside the class; the only way to create an instance of the class is by calling a static method within the class which create an instance or returns a reference to an existing instance. Generally , they are used in singleton design patterns, where the code ensures that only one instance of a class can ever be created.
- **Static Constructor** :- A static constructor is used to initialize any static data, or to perform a particular action that needs to be performed once only. It is called automatically before the first instance is created or any static members are referenced.

Key Points :-

- A Class have any number of constructors.
- A constructor doesn't have any return type even void.
- A static constructor can't be parameterized constructor.
- Within a class you can create only one static constructor.
- A Static constructor can't be inherited and called be directly.

8. Encapsulation : - Encapsulation is a moderately easy concept once we realise it comes from the same base word as **capsule**. It simple a containment of information. It means putting things, closing them in a package and the 'things' we are talking about here are data and functions. Classes, properties and access modifiers are tools to provide encapsulation in c#. Encapsulation reduce the complexity other aspect abstraction in order to provide reusability. The Object-oriented principle of Encapsulation helps avoid such problems, allowing you to hide internal state and abstract access to it through type members such as methods, properties and indexers. Encapsulation helps you reduce coupling between objects and increases the maintainability of our code. Encapsulation is implemented by using access specifiers. An access defines the scope and visibility of a class members. C# supports public, private, protected and internal access specifiers. Basically it is the process of combining data and functions into a single unit called class. In encapsulation, the data is not accessed directly; it is accessed through the functions present inside the class. In simpler words, attributes of the class are kept private and public getter and setter methods are provided to manipulate these attributes. Thus, encapsulation makes the concepts of data hiding possible.

In other word Encapsulation is Object Oriented Programming Concepts that binds together data and functions that manipulate the data, and that keeps both safe from outside interface and misuse. Data Encapsulation led to the important OOP concept of data hiding.

9. Abstraction : - Abstraction is a process where you show only "relevant" data and "hide" unnecessary details of a object from the user. It is generalised term. It's also a powerful methodology to manage complex systems. Abstraction is managed by well-defined objects and their hierarchical classification. Abstraction is used for reduce complexity it provide a simple interface to use. Through simple interface it support polymorphism. Abstraction is enabled by encapsulation, because we encapsulate objects.

Abstraction Use ~ Encapsulation & Encapsulation use ~ Data Hiding

10. Polymorphism : - Polymorphism means one name many forms, Polymorphism means one object behaving as multiple forms. One function behaves in different forms. In other words, "Many forms of a single object is called polymorphism." There are two type of polymorphism.

- **Static or Compile time polymorphism** :- In compile time polymorphism is method overloading. Where the compiler knows which overloaded method it is going to call. Method overloading is a concept where a class can have more than one method with the same name and different parameters. Compile check the type and number of parameters passed on to the method and decides which method to call at compile time and it will give an error if there are no methods that match the method signature of the method that is call at compile time. Operator overloading is also compile time polymorphism. It is also called as **Early Binding or Static Binding**.
- **Dynamic or Runtime polymorphism** :- Runtime polymorphism is achieved by method overriding. Method overriding allows us to have methods in the base and derived classes with the same name and the same parameters. We can point to any derived class from the object of the base class at runtime that shows the ability of runtime binding. Through the reference variable of a base class the determination of the method to be called is based on the object being referred to be reference variable. Compiler would not be aware whether the method is available for overriding the functionality or not. So compiler would not give any error at compile time. Virtual method is a method behavior can be overridden in derived class. Virtual method allows declare a method in base class that can be redefined in each derived class. When a virtual method is invoked, the runtime type of the object is checked for an overriding member. The overriding member in the most derived class is called which might be the original member. By default methods are non-virtual.

Key Points :-

- i. We can't override non-virtual method.
- ii. We can't use the virtual modifier with the **static, abstract, private or override** modifier.
- iii. Virtual properties behave like **abstract** methods, except for the differences in declaration and invocation syntax.
- iv. It is an error to use the **virtual** modifier on a **static** property.

11. Interface :- An interface looks like a class, but has no implementation. The only things it contains are declaration of events, indexers, methods and properties. The reason interfaces only provide declarations is because they are inherited by structs and classes, that must provide an implementation for each interface member declared. Interfaces in c# provide a way to achieve runtime polymorphism. Using interface we can invoke functions from different classes through the same interfaces reference, whereas using virtual functions we can invoke functions from different classes in the same inheritance hierarchy through the same reference.

12. What are Modifier in C#?

- **Abstract :-** The abstract modifier is used for a method or property in a class that has no implementation or for a class that contains such method. A class with abstract member cannot be instantiated with the new operator.
- **Sealed :-** The sealed modifier is used for prevent derived class to overriding specific virtual method and properties.
- **Virtual :-** Virtual modifier are an implementation of type-based polymorphism. When we have a derived class and a base class the derived class can re-implement the base virtual method. We can't use virtual modifier with the static, abstract, private and override modifier. It will be compile time error.
- **Const :-** Specifies that the value of field or the local variable can't be modified.
- **Event :-** Declare a events.
- **Extern :-** Indicates that the method is implemented externally.
- **Override :-** Provide a new implementation of a virtual member inherited in base class.
- **Partial :-** Define partial classes, struct and method throughout assembly.
- **Static :-** This modifier call itself without any objects
- **Unsafe :-** Declared unsafe context
- **Volatile :-** Means of modifier is make sure that the compiler and the JIT do not perform any recording or register caching optimization on this variable. It also means tell the processors to do whatever it is they need to do ensure that i am reading the latest value, even if that means halting other process and making them synchronize main memory with their caches. It essential means that threads should not cache such variable or in other words should not trust the values of these variables unless they are directly read from the main memory.
- **New :-** To hide an inherited member, declare it in the derived class by using the same member name and modify it with the new keyword. Override can't be use with new.
- **ReadOnly :-** Declares a field that can be assigned value as part of the declaration or in a constructor in same class.

13. What are Access-Modifier in C#?

- **Public:-** Access is not restricted.
- **Private :-** Private access is the least permissive access level. Private member are accessible only within the body of the class or the struct in which they are declared.
- **Protected :-** A protected member is accessible within its class and by derived class instance. A protected member of a base class is accessible in a derived class only if the access occurs through the derived class type. if we access
- **Internal :-** Access is limited to the current assembly but not access from outside assembly. That is default access modifier for class.
- **Protected Internal :-** Access is limited to the current assembly or types derived from the containing class.

14. Difference between encapsulation and abstraction.

ANS:

1. Abstraction is code level and Encapsulation is for application level. Application is ready to use we know only button level, data etc but we don't know internal functionality of particular application that is called Encapsulation.but in the code like when we are creating any dll that time who will use dll for that user that is called Abstraction.
2. Encapsulation is accomplished by using Class. - Keeping data and methods that accesses that data into a single unit"
- "Abstraction is accomplished by using Interface. - Just giving the abstract information about what it can do without specifying the background details.

Abstraction is a process where you show only "relevant" data and "hide" unnecessary details of an object from the user. Consider your mobile phone, you just need to know what buttons are to be pressed to send a message or make a call, What happens when you press a button, how your messages are sent, how your calls are connected is all abstracted away from the user.

Encapsulation is the process of combining data and functions into a single unit called class. In Encapsulation, the data is not accessed directly; it is accessed through the functions present inside the class. In simpler words, attributes of the class are kept private and public getter and setter methods are provided to manipulate these attributes. Thus, encapsulation makes the concept of data hiding possible.**Encapsulation is a means of hiding details in order to protect them from outside manipulation.**

15. Difference between Lazy Loading, Eager Loading and Explicit Loading.

Eager Loading :- When you are sure that want to get multiple entities at a time, for example you have to show user, and user details

at the same page, then you should go with eager loading. Eager loading makes single hit on database and load the related entities.

Lazy Loading :- When you have to show users only at the page, and by clicking on users you need to show user details then you need to go with lazy loading. Lazy loading make multiple hits, to get load the related entities when you bind/iterate related entities.

Explicit Loading :- There are options to disable Lazy Loading in an Entity Framework. After turning Lazy Loading off, you can still load the entities by explicitly calling the Load method for the related entities. There are two ways to use Load method Reference (to load single navigation property) and Collection (to load collections). When you have turned off Lazy Loading, use Explicit loading when you are not sure whether or not you will be using an entity beforehand.

In C#, the memory allocation for primitive data types depends on whether they are value types or reference types:

- **Value Types:** These include primitive data types like `int`, `float`, `bool`, and `structs`. They are typically stored on the stack. [For example, if you declare an `int` inside a method, it will be allocated on the stack.](#)
- **Reference Types:** These include types like `class`, `array`, and `string`. They are stored on the heap. [When you create an instance of a class, the object itself is stored on the heap, but the reference to that object is stored on the stack.](#)